

Bridging Imperative Process Models and Process Data Queries—Translation and Relaxation

Abdur Rehman Anwar Qureshi¹, Adrian Rebmann², Timotheus Kampik^{2,3},
Matthias Weidlich⁴, and Mathias Weske¹

¹ Hasso Plattner Institute, University of Potsdam, Potsdam, Germany

`{rehman.qureshi|mathias.weske}@hpi.de`

² SAP Signavio, Berlin, Germany

`{adrian.rebmann|timotheus.kampik}@sap.com`

³ Umeå University, Umeå, Sweden

⁴ Humboldt Universität zu Berlin, Germany

`matthias.weidlich@hu-berlin.de`

Abstract. Business process management is increasingly practiced using data-driven approaches. Still, classical imperative process models, which are typically formalized using Petri nets, are not straightforwardly applicable to the relational databases that contain much of the available structured process execution data. This creates a gap between the traditional world of process modeling and recent developments around data-driven process analysis, ultimately leading to the under-utilization of often readily available process models. In this paper, we close this gap by providing an approach for translating imperative models into relaxed process data queries, specifically SQL queries executable on relational databases, for conformance checking. Our results show the continued relevance of imperative process models to data-driven process management, as well as the importance of behavioral footprints and other declarative approaches for integrating model-based and data-driven process management.

Keywords: Process Querying · Declarative Models · Conformance

1 Introduction

Imperative process models, such as those specified using the Business Process Model and Notation (BPMN) standard [15], have emerged as the standard approach for modeling and communicating about operational processes. These models provide an intuitive abstraction of reality and require moderate training to create. However, the shift from model-driven business process management (BPM) to data-driven BPM has prompted growing interest in declarative approaches to process specification [9]. This trend is driven by the inherent flexibility of the declarative paradigm, which often better aligns with the complexity and variability of real-world processes. Moreover, declarative constraints, which are the central ingredient of these specifications, can be readily translated into queries in well-established languages, such as SQL, that can then be directly executed on process execution data, enabling immediate data-driven insights [9,4].

A key process analysis task that highlights the value of the declarative approach is conformance checking. Organizations rely on conformance checking to ensure that operational processes align with desired behavior. While imperative models provide a convenient way to represent this behavior, interpreting it in terms of queries allows for more direct and flexible conformance analysis. In particular, conformance checking with real-world imperative process models tends to overwhelm the user with many deviations, due to the models’ overly rigid nature that does not reflect a potentially more relaxed *intended* meaning. For instance, while an invoice should ideally be checked after it has been received, real-world processes might involve intermediate steps such as asking for clarification or responding to a vendor inquiry. Declarative approaches naturally capture such flexibility, while allowing for immediate and efficient checking of conformance based on queries. This dual need to (1) retain the convenience of imperative modeling while (2) leveraging the flexible querying capabilities of declarative approaches underscores the importance of bridging the two paradigms.

This bridge, however, is not straightforward. A clear understanding of how the behavior specified in imperative models can be translated into process data queries is essential but underexplored. Although previous studies [2,3,12] have translated imperative models into declarative specifications, these efforts lack a demonstration of the relationship to queries and do not consider a systematic relaxation of constraints for conformance checking. This limits the ability to seamlessly integrate imperative models and process data querying.

In this paper, we address this gap through a structured approach to bridge imperative process models and queries. Specifically, we leverage the trace semantics of sound, free-choice workflow nets, representing them through the well-known directly-follows relations [18]. Based on these, a user can systematically relax the allowed behavior by changing relations to accommodate the flexibility of real-world processes thus allowing for deviations that remain logically consistent with the *intended* behavior of the original model. The remaining relations provide the foundation for deriving a corresponding set of declarative constraints, naturally translate into process data queries (e.g., in plain SQL), which can be directly executed on event data for conformance checking.

The remainder is structured as follows. Section 2 presents a motivating example to illustrate the practical problem we address. Section 3 introduces the concepts our work is built on. Section 4 details our approach, explaining how we determine and relax behavioral relations to generate declarative constraints. Section 5 provides a proof of concept by applying our approach on real-world data. Section 6 discusses related research and Section 7 concludes the paper.

2 Motivating Example

In this section, we illustrate the value of translating imperative process models into more relaxed process data queries using the process model in Figure 1.

The process model outlines a procurement workflow, beginning with the creation and approval of a purchase requisition, which triggers the generation of a

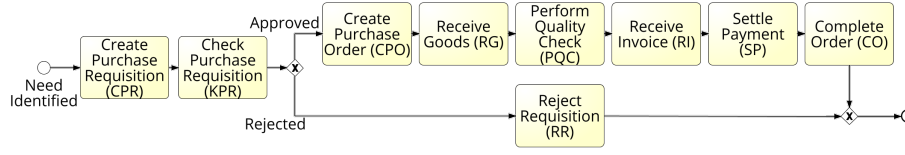


Fig. 1: An exemplary BPMN process model of a procurement process.

purchase order. Following this, the model requires that goods must be received and then pass a mandatory quality check. Only after this inspection is complete can the invoice be received, leading to the final step of settling the payment.

Table 1: One case of the process for a standard office supply purchase.

Timestamp	Activity	Notes
2023-11-10 10:00:00	Create Purchase Requisition	Order for standard printer paper
2023-11-10 11:30:00	Approve Purchase Requisition	Auto-approved due to low value
2023-11-10 11:35:00	Create Purchase Order	
2023-11-13 14:00:00	Receive Goods	1 box of printer paper arrived
2023-11-13 15:10:00	Receive Invoice	Invoice received via email
2023-11-20 09:00:00	Settle Payment	

In real-world settings, processes rarely adhere perfectly to such rigid specifications. There are often exceptions, for instance, to enhance efficiency. Consider the event log for a single case in Table 1, which details the purchase of standard office supplies from a trusted vendor. For a low-value, routine item, performing a formal, documented quality check is inefficient and unnecessary. The process owner has thus allowed this step to be skipped to save time and resources. However, when a conformance checking approach aligns the event log in Table 1 to the BPMN model in Figure 1, it will detect deviations, namely skipping the purchase requisition and quality checks and the order completion.

A possible solution is to shift from an imperative to a declarative paradigm. Instead of updating the rigid model for every exceptional situation that may occur, a declarative model that specifies a more relaxed set of constraints is used. For instance, from the model the following relaxed rule set can be derived: (1) *Create Purchase Requisition* must occur, (2) *Check Purchase Requisition* must precede *Approve Purchase Requisition*, (3) *Receive Invoice* must follow *Receive Goods* (4) *Perform Quality Check* is optional, however, if it occurs, it must occur after *Receive Goods* and before *Settle Payment* (5) *Settle Payment* must occur after *Receive Invoice*. Checking conformance of the case in Table 1 against these rules, none of them are violated. In particular, also Rule 4 is not violated because its precondition, the occurrence of *Perform Quality Check*, is not met.

Imperative models, while crucial for designing processes, are often too rigid for real-world conformance checking, flagging any exception as a deviation. The declarative paradigm focuses on core constraints over predefined paths, offering more flexible conformance checking.

3 Preliminaries

Sound, Free-Choice Workflow Net. A workflow net (WF-net) is a specific type of Petri net. Formally, it is a tuple $W = (P, T, F)$, where P is a set of places, T is a set of transitions, and F is the flow relation between them. To qualify as a WF-net, the structure must satisfy three key properties (1) *A Single Start Point*: The net must contain one unique *source* place i that has no incoming transitions ($\bullet i = \emptyset$). (2) *A Single End Point*: It must also contain one unique *sink* place o that has no outgoing transitions ($o \bullet = \emptyset$). (3) *Full Connectedness*: Every other node (place or transition) must lie on a path from the start place i to the end place o . A *sound, free-choice workflow net* adds two crucial guarantees. *Soundness* ensures that the process can always complete properly, without deadlocks or leftover tasks. *Free-choice* ensures that when multiple transitions compete for tokens from the same place, that choice is self-contained and not dependent on other parts of the workflow net.

Behavioral Relations. Alpha-relations offer a fundamental abstraction of process behavior, applicable to both event logs and process models, and have been a cornerstone of process mining since its inception. When inferred from sound, free-choice workflow nets, they describe behavioral relationships between two activities $a, b \in A$ as follows: *Directly-follows* ($a \rightarrow b$): Activity a can be immediately followed by activity b . *Concurrency* ($a \parallel b$): Activities a and b can occur in any order. *Choice* ($a - b$): Activities a and b never follow each other directly.

The transitive closure of the directly-follows relation ($\rightarrow$) links directly-follows ($\rightarrow$) and eventually-follows ($\prec$) relations. This means that $p \prec q$ if there exists a path of one or more *directly-follows* arrows leading from p to q . For instance, if $p \rightarrow a$, $a \rightarrow b$, and $b \rightarrow q$, then $p \prec q$. We write $\text{TC}(D)$ for the function that computes the transitive closure of directly-follows relations in D , which means it identifies all pairs of activities (p, q) where q can be eventually reached from p through a sequence of directly-following activities.

From Workflow Net to Directly-Follows Relations. For sound, free-choice workflow nets, it has been shown that the directly-follows (there called *1-successor*) relations can be inferred directly from the net's structure [18]. The approach is based on the concept of the Minimal Structural Successor (MSS) function, which for any two transitions x and y , identifies the minimal set of nodes structurally required to enable y after x has fired. As proven in their work, y is a 1-successor of x if and only if the corresponding MSS set contains exactly one transition, namely the source transition x itself, formally $|mss(x, y) \cap T| = 1$. This guarantees that no other transition is required in between x and y . Therefore, by calculating the MSS for all transition pairs, the directly-follows relations that define the same trace semantics as the net can be determined without exhaustive trace generation or state-space exploration.

Declarative Constraints. We translate behavioral relations into declarative constraints focusing on (*Branched*) *Declare* [16], which is a language that abstracts from *Linear Temporal Logic on Finite Traces* (LTL_f) [8]. It allows us to specify constraints that must hold over the sequence of activities and is built

on several core temporal operators, where ϕ represents a logical condition $\mathbf{G}\phi$ (Globally): ϕ must hold at every step of the trace. $\mathbf{X}\phi$ (Next): ϕ must hold in the immediately following step. $\phi_1 \mathbf{U} \phi_2$ (Until): ϕ_1 must hold until ϕ_2 eventually holds. $\mathbf{F}\phi$ means that ϕ holds at some point in the future. Table 2 summarizes the semantics of selected *Declare* templates, whose parameters are sets. This means that any one of those activities can activate resp. satisfy the constraint. For example, the template `AlternateResponse`($\{a, b\}, \{c\}$) defines that if activity a or ($\vee$) activity b occurs, c must happen afterwards. This provides more flexibility than standard *Declare*.

Table 2: Semantics of the declarative constraints

Constraint	LTLf formula	Description
<code>Init</code> ($\{p_1, \dots, p_n\}$)	$\mathbf{G}(\text{start} \rightarrow \mathbf{X}(\{p_1 \vee p_2 \vee p_n\}))$	Assert that at least one activity from $\{p_1, p_2, \dots, p_n\}$ occurs in start.
<code>ChainResponse</code> ($\{p_1, \dots, p_n\}, \{q_1, \dots, q_n\}$)	$\mathbf{G}(\{p_1 \vee \dots \vee p_n\} \rightarrow \mathbf{X}(\{q_1 \vee \dots \vee q_n\}))$	Whenever at least one activity from $\{p_1, \dots, p_n\}$ occurs, then one of $\{q_1, \dots, q_n\}$ must occur in the immediately following step.
<code>AlternateResponse</code> ($\{p_1, \dots, p_n\}, \{q_1, \dots, q_n\}$)	$\mathbf{G}(\{p_1 \vee \dots \vee p_n\} \rightarrow \mathbf{X}(\neg\{p_1 \vee \dots \vee p_n\} \mathbf{U} \{q_1, \vee \dots \vee, q_n\}))$	Whenever at least one activity from $\{p_1, \dots, p_n\}$ occurs, then one of $\{q_1, \dots, q_n\}$ occurs in afterwards, and not a single activity from $\{p_1, \dots, p_n\}$ recurs in between.

4 Approach

In this section, we give an overview of our approach, before detailing its steps.

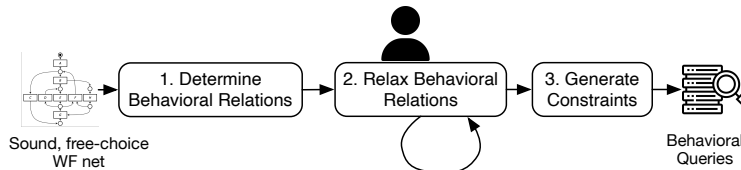


Fig. 2: Approach overview.

4.1 Approach Overview

We present a high-level overview of our approach in Figure 2. It takes as input a sound, free-choice WF-net (as defined in Section 3), which can be obtained from a BPMN diagram using the established method by Dijkman and Ouyang [11]. The approach then consists of three steps: (1) determining behavioral relations, (2) relaxing behavioral relations (done by a user), and (3) generating declarative constraints. The output is a set of behavioral process data queries that can be straightforwardly created from the declarative constraints.

4.2 Step 1: Determine Behavioral Relations

Step 1 begins by determining directly-follows relations from a sound, free-choice workflow net provided as input. To this end, it employs the method presented in [18] (detailed in Section 3). Subsequently, based on these directly-follows relations, Step 1 derives Alpha-relations. The output of Step 1 is a relation matrix, as shown in Table 3 for the workflow net from our running example. Crucially, these resulting relations preserve the same trace semantics as the input net.

Table 3: Behavioral relation matrix of the running example.

	CPR	KPR	CPO	RG	PQC	RI	SP	CO	RR
CPR	-	$\rightarrow$	-	-	-	-	-	-	-
KPR	$\leftarrow$	-	$\rightarrow$	-	-	-	-	-	$\rightarrow$
CPO	-	$\leftarrow$	-	$\rightarrow$	-	-	-	-	-
RG	-	-	$\leftarrow$	-	$\rightarrow$	-	-	-	-
PQC	-	-	-	$\leftarrow$	-	$\rightarrow$	-	-	-
RI	-	-	-	-	$\leftarrow$	-	$\rightarrow$	-	-
SP	-	-	-	-	-	$\leftarrow$	-	$\rightarrow$	-
CO	-	-	-	-	-	-	$\leftarrow$	-	-
RR	-	$\leftarrow$	-	-	-	-	-	-	-

4.3 Step 2: Relax Behavioral Relations

Next, we introduce types of operations that a process analyst can apply to perform relaxations of the initial behavioral relations of the input model. Once a relaxation operation is applied, our approach updates the matrix accordingly. In this manner, it is transparent how other behavioral relations are affected by the relaxation, which allows a user to judge if this is reasonable for their process analysis and make adjustments if needed.

(1) Remove Activity. This operation makes an activity optional, while also allowing that it can appear whenever. This enables a user to accommodate context-dependent steps. As seen in the running example, skipping the *Perform Quality Check* for a routine purchase is a valid efficiency gain. This prevents such cases from being flagged as deviations. This is achieved by setting all its relations with other activities to *undirected, eventually-follows* (denoted as $\prec\triangleright$).

(2) Remove All Relationships Between Two Activities. This operation removes all ordering relations between two activities, making them completely independent. This is useful when process fragments are less coupled than the original model suggests. For instance, if a model enforces *Receive Goods* $\rightarrow$ *Receive Invoice*, this relaxation allows to accommodate real-world scenarios where the invoice arrives before the goods, e.g., when the two packages are sent by different departments. Their relationship is set to $\prec\triangleright$ thereby allowing (but not forcing) them to occur in any order both directly and indirectly.

(3) Turn Exclusive (-) into Direct Relationship ($\rightarrow$). This operation replaces a choice between activities ($A - B$) with a strict sequence ($A \rightarrow B$). This

covers scenarios where activities, once considered exclusive, can co-exist. For instance, if a process allows either an *Automated Approval* or a *Manual Approval*, this relaxation would handle cases where *both* occur for auditing purposes, preventing the system from flagging such valid behavior as a deviation. Note that this also allows for an activity to be followed by itself (e.g., introducing $A \parallel A$), a relaxation essential for modeling rework or iterative tasks. For instance, if for a *Purchase Order* several *Invoices* are created, a rigid model would flag the resulting self-loop as a deviation. This relaxation considers such behavior as an acceptable part of the process, thereby reducing conformance noise.

(4) Turn Direct ($\rightarrow$) into Indirect Relationship ($\prec$). This operation allows that B can also indirectly follow A . This is critical for conformance, as real-world processes often contain unmodeled steps. For example, an employee might *Log Goods in System* between *Receive Goods* and *Perform Quality Check*. By removing the direct relationship, the behavior gains the flexibility to accept such valid traces.

4.4 Step 3: Generating Declarative Constraints

After the user applied a series of relaxation operations, the approach automatically generates a set of declarative constraints as follows.

First, our approach creates a set of directly-follows relations D and a set of eventually-follows relations E based on the (relaxed) matrix. All relations are represented as pairs (a, b) of activities, where bidirectional relations become two individual relations $((a, b)$ and $(b, a))$. Then, it generates constraints as outlined in Algorithm 1: First, the algorithm initializes the constraint set with one **Init** constraint using the **getStartActivities** function. It determines the set of start activities, I , by identifying all activities $a \in A$ that are never a successor, i.e., $I = \{a \in A \mid \forall b \in A, (b, a) \notin D\}$. Then the algorithm continues with generating binary constraints. Its default template is **ChainResponse**, as it models the directly-follows relation by enforcing that if an activity a occurs, it must be immediately followed by one activity of a given set of successors (lines 4 and 6). However, this constraint’s inherent OR-logic is too permissive to model parallel behavior, where multiple activities must all occur following a predecessor. For these cases, the algorithm switches to generating separate **AlternateResponse** constraints for each parallel activity, which correctly enforces that the predecessor’s occurrence requires the eventual occurrence of each successor (lines 7–16). This step is further refined to handle optional tasks. To avoid wrongly enforcing a parallel activity that can be skipped, the algorithm employs the **ISOPTIONALACTIVITY** function (lines 8 and 11). Before creating an **AlternateResponse** constraint, it checks if the target activity can be bypassed and—if so—does not create the constraint, correctly preserving the activity’s optionality.

The Boolean function **ISOPTIONALACTIVITY** takes an activity x and the directly-follows relations D as input. It returns *true* if (and only if) x is identified as optional (skipable) based on a bypass pattern: for every direct predecessor a of x and every successor b of x that are not in parallel relation with x and each other, i.e., $(x, a) \notin D \wedge (b, x) \notin D \wedge (b, a) \notin D$, it checks if any $(a, b) \in D$.

Finally, the algorithm processes the eventually-follows relations in the set E to generate **AlternateResponse** constraints if these are not already covered by the transitive closure of the directly-follows relations (lines 17–21).

Algorithm 1: Generate constraints

Require: $D, E \subseteq A \times A$, sets of directly-follows and eventually-follows relations (respectively)
Ensure: C , a set of declarative constraints

```

1:  $C \leftarrow \{\text{Init}(\text{GETSTARTACTIVITIES}(D))\}$ 
2:  $A_D \leftarrow \{a \mid (a, b) \in D\}$   $\triangleright$  Get unique source activities for direct-follows
3: for each  $a \in A_D$  do
4:    $S \leftarrow \{x \mid (a, x) \in D\}$   $\triangleright$  Collect all directly-following activities
5:    $C \leftarrow C \cup \{\text{ChainResponse}(\{a\}, S)\}$   $\triangleright$  Generate ChainResponse for  $a$ 
6:   for each  $\{b, c\} \subseteq S$  do
7:     if  $(b, c) \in D \wedge (c, b) \in D$  then  $\triangleright$  Generate dedicated AlternateResponse
        constraints for parallel relations
8:       if  $\neg \text{IsOptionalActivity}(b, D)$  then
9:          $C \leftarrow C \cup \{\text{AlternateResponse}(\{a\}, \{b\})\}$ 
10:      end if
11:     if  $\neg \text{IsOptionalActivity}(c, D)$  then
12:        $C \leftarrow C \cup \{\text{AlternateResponse}(\{a\}, \{c\})\}$ 
13:     end if
14:   end if
15: end for
16: end for
17:  $A_E \leftarrow \{a \mid (a, b) \in E\}$   $\triangleright$  Get unique source activities for eventually-follows
18: for each  $a \in A_E$  do
19:    $S \leftarrow \{x \mid (a, x) \in E \wedge (a, x) \notin \text{TC}(D)\}$   $\triangleright$  Collect successors from  $E$  that are
        not in the transitive closure of  $D$ 
20:    $C \leftarrow C \cup \{\text{AlternateResponse}(\{a\}, S)\}$   $\triangleright$  Generate AlternateResponse for  $a$ 
21: end for
22: return  $C$ 

```

Selected declarative constraints and corresponding SQL queries for the running example (no relaxation) are presented in Table 4. The SQL queries can straightforwardly be created from the *Declare* constraints (cf. [4] for details) assuming the following database schema: an event log is represented in a single *events* table, where each row represents a distinct event. The table has three columns: the *case_id* serves to group events into process instances, the *end_time* timestamp orders the events, and the *event_name* provides the activity label.

5 Proof of Concept

This section demonstrates the practical value of our approach in a conformance checking scenario. We apply our approach to a model of a real-world process

Table 4: Selection of declarative constraints and corresponding SQL fragments. All queries use `SELECT case_id FROM events MATCH_RECOGNIZE, PARTITION BY case_id ORDER BY end_time, and ONE ROW PER MATCH.`

Constraint	SQL Fragment (PATTERN and DEFINE)
Init({CPR})	PATTERN (^CPR ANY*) DEFINE CPR AS event_name = 'CPR'
ChainResponse({KPR}, {CPO,RR})	PATTERN (ANY* KPR (CPO RR) ANY*) DEFINE KPR AS event_name = 'KPR', CPO AS event_name = 'CPO', RR AS event_name = 'RR'
ChainResponse({RI}, {SP})	PATTERN (ANY* RI SP ANY*) DEFINE RI AS event_name = 'RI', SP AS event_name = 'SP'

and its corresponding public event log. To validate the benefits of our novel relaxation step, we compare the resulting conformance insights against traditional alignment-based conformance checking.

Implementation. We implemented the proposed approach as a Python-based command-line interface. The source code is publicly available.⁵

Data. For our proof of concept, we use the BPI Challenge 2019 (BPIC19) dataset⁶. We selected the *3-way matching, invoice before goods receipt* model from the winning submission by Diba et al. [10]; it covers 86.36% (217,406) of the 251,734 cases in the original event log. To prepare this model for our analysis, we first converted it into a sound, free-choice workflow net—the required input format for our approach—using the method by Dijkman and Ouyang [11].

Setup. We check conformance using queries that we obtain when applying our approach with relaxation to the BPIC19 workflow net and compare the results with *Alignments*, a state-of-the-art conformance-checking technique [1], which aligns event log traces to an imperative process model, i.e., the same net from which our approach generates queries.

Approach Application. We apply our approach to generate behavioral relations, relax these, and generate declarative constraints that map to SQL queries. Based on the initial behavioral relation matrix (after Step 1), we apply the following relaxations for the purpose of this proof-of-concept:

- *Recurring Invoicing and Deliveries:* To reflect valid business patterns where multiple invoices may precede goods receipt, or multiple deliveries occur before a single invoice, we first relax “Record Invoice Receipt” and “Record Goods Receipt” by turning direct relation ($||$) into indirect one ($\prec\triangleright$), via relaxation type (4), which allows for these to reoccur or be interleaved.

⁵ <https://github.com/rehman-queeshi/bridging-models-and-queries>

⁶ <https://icpmconference.org/2019/icpm-2019/contests-challenges/bpi-challenge-2019>

- *Decoupling Early Procurement Stages:* We then loosen constraints in the initial stages of the procurement process to allow for greater independence. Specifically, we remove the relationship (relaxation type (2)) between “Create Purchase Order Item” and “Record Order Confirmation” to allow for independent order confirmation. Furthermore, to permit flexible adjustments of price and quantity before goods movement and invoice handling, we remove all relations between “Change Price”, “Change Quantity”, and related purchase requisition/order activities (“Create Purchase Requisition Item”, “Create Purchase Order Item”, “Record Order Confirmation”).

These relaxations result in behavioral relations that, after translated into constraints and finally queries, considerably reduce the total number of violations by allowing for behavior that is reasonable, allowing analysts to focus on violations of rules that actually matter to the analyst.

Results. Table 5 reports the conformance rate (# of traces conforming to the model resp. all constraints divided by total # of traces) for both approaches.

Table 5: Conformance rates.

Approach	Conformance Rate
Alignments	0.691
Relaxed queries	0.823

The numbers illustrate the substantial decrease of detected deviating traces after relaxation, reducing the cognitive load of users that need to investigate which of these are indeed problematic.

6 Related Work

Our work primarily relates to the translation of imperative process models into declarative specifications, relaxation in conformance checking, declarative conformance checking, and the direct querying of process data.

From Imperative to Declarative Models. Previous work has focused on extracting declarative constraints from BPMN diagrams [3] and Petri nets [12]. Barbaro et al. present the first approach to systematically turn safe and sound Workflow nets into declarative specifications [2]. While their goal is to achieve equivalence of the trace semantics of both models, the goal of our approach is to provide a means for systematically relaxing the allowed behavior instead.

Relaxation in Conformance Checking. A certain degree of relaxation can also be realized within alignment-based conformance checking by manipulating the cost function [6]. By setting the cost of a log move for an activity to zero, that activity is ignored (optional). While this can implement some simple relaxations, its capabilities are limited. For instance, if two formerly exclusive activities should now be allowed to co-occur, we would require a context-sensitive cost function. This would mean the cost of a log move for one activity would be

zero only if the other activity had already occurred in the trace. Such dynamic, context-aware costing is not achievable with a traditional, static cost function.

Declarative Conformance Checking. The use of declarative languages like *Declare* [17] for conformance checking has gained significant traction for its ability to handle process flexibility. Methods range from providing a simple boolean (conforming/non-conforming) result [7] to reporting detailed statistics, such as activations, satisfactions, and violations, for each constraint and trace [5,14,13].

Process Querying using SQL. The direct analysis of event data stored in relational databases is a growing area of interest. Generating SQL queries as done in our approach, builds on applications of native SQL features—specifically, the `MATCH_RECOGNIZE` clause—to pattern matching in event logs [4].

Overall, prior work has not systematically bridged imperative models with process data querying. Our approach fills this gap by translating imperative models into executable queries with user-driven relation relaxation, enabling more flexible and practical conformance checking.

7 Conclusion

We presented a structured approach to translate sound, free-choice workflow nets into declarative constraints and process data queries based on their underlying behavioral relations, bridging imperative models and data-driven analysis. By allowing users to systematically relax the relations, our approach accommodates real-world process flexibility and reduces conformance checking noise. Our proof-of-concept shows a considerable reduction in violations, which enables analysts to focus on genuine non-conformance. This work highlights the continued relevance of imperative models as a source of process knowledge while leveraging the flexibility of declarative models for scalable data-driven analysis.

While the main purpose of transforming the behavioral matrix into declarative constraints is to ease subsequent query relaxation, a notable limitation is that it remains to be demonstrated whether or not the resulting constraints—without prior relaxation—are guaranteed to be behaviorally equivalent to the original process model. Especially the presence of *silent transitions*, which are common in real-world process models poses a challenge. Our current approach does not fully address the issues posed by these transitions, with the exception of accounting for optional activities (skip steps) during the translation of relations into constraints. Further, we acknowledge the limited scope of our relaxation operations, which can be extended to a more comprehensive set. The validation of our approach was conducted on a single dataset and a more comprehensive evaluation is needed to generalize the findings. Furthermore, the approach’s value has not been validated with users, which is an important area for future research.

Future research shall address the limitations outlined above. Further potential directions for future work include the expansion of relaxation operations, the development of a visual interface to guide domain experts through the relaxation process, and the extension of the translation mechanism to encompass a broader range of process models beyond free-choice workflow nets.

References

1. Van der Aalst, W., Adriansyah, A., Van Dongen, B.: Replaying history on process models for conformance checking and performance analysis. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* **2**(2), 182–192 (2012)
2. Barbaro, L., Varricchio, G., Montali, M., Di Ciccio, C.: From sound workflow nets to ltl f declarative specifications by casting three spells. In: *International Conference on Business Process Management (Forum)*. pp. 3–22. Springer (2025)
3. Bergmann, A., Rebmann, A., Kampik, T.: Bpmn2constraints: Breaking down BPMN diagrams into declarative process query constraints. In: *BPM Demos and Resources. CEUR Workshop Proceedings*, vol. 3469, pp. 137–141 (2023)
4. Brand, J., Kampik, T., Okulmus, C., Weidlich, M.: One language to rule them all: Behavioural querying of process data using SQL. In: Delgado, A., Slaats, T. (eds.) *ICPM Workshops, Lyngby*, pp. 18–30. Springer (2024)
5. Burattin, A., Maggi, F.M., Sperduti, A.: Conformance checking based on multi-perspective declarative process models. *Expert systems with applications* **65**, 194–211 (2016)
6. Carmona, J., Van Dongen, B., Solti, A., Weidlich, M.: Conformance checking. Switzerland: Springer.[Google Scholar] **56**, 12 (2018)
7. Chiariello, F., Maggi, F.M., Patrizi, F.: Asp-based declarative process mining. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 36, pp. 5539–5547 (2022)
8. De Giacomo, G., Vardi, M.Y., et al.: Linear temporal logic and linear dynamic logic on finite traces. In: *Ijcai*. vol. 13, pp. 854–860 (2013)
9. Di Ciccio, C., Montali, M., et al.: Declarative process specifications: Reasoning, discovery, monitoring. *Process mining handbook* **448**, 108–152 (2022)
10. Diba, K., Remy, S., Pufahl, L.: Compliance and performance analysis of procurement processes using process mining. In: *BPM Workshops* (2019)
11. Dijkman, R.M., Dumas, M., Ouyang, C.: Semantics and analysis of business process models in bpmn. *Information and Software technology* **50**(12), 1281–1294 (2008)
12. Goulart Rocha, E., van Zelst, S.J., van der Aalst, W.M.: Mining behavioral patterns for conformance diagnostics. In: *International Conference on Business Process Management*. pp. 291–308. Springer (2024)
13. Maggi, F.M., Montali, M., Bhat, U.: Compliance monitoring of multi-perspective declarative process models. In: *2019 IEEE 23rd International Enterprise Distributed Object Computing Conference (EDOC)*. pp. 151–160. IEEE (2019)
14. Montali, M., Maggi, F.M., Chesani, F., Mello, P., Aalst, W.M.v.d.: Monitoring business constraints with the event calculus. *ACM transactions on intelligent systems and technology (TIST)* **5**(1), 1–30 (2014)
15. OMG: Business Process Model and Notation (BPMN), Version 2.0 (January 2011), <http://www.omg.org/spec/BPMN/2.0>
16. Pesic, M.: Constraint-based workflow management systems: shifting control to users. Phd thesis, TU/e Industrial Engineering and Innovation Sciences (2008)
17. Pesic, M., Schonenberg, H., Van der Aalst, W.M.: Declare: Full support for loosely-structured processes. In: *EDOC*. pp. 287–287. IEEE (2007)
18. Weidlich, M., van der Werf, J.M.E.M.: On profiles and footprints - relational semantics for petri nets. In: *PETRI NETS. Lecture Notes in Computer Science*, vol. 7347, pp. 148–167. Springer (2012)